Learning and Optimizing XGBoost with Planned Experiments

Accuracy, Prediction Speed, and Comparison with Adaptive Search

Christos Chousein Sounios

October 12, 2026

Abstract

Can a structured, pre-planned experiment both explain and optimize an XGBoost model, and how does that approach compare with adaptive search? We studied four hyperparameters on the California Housing regression task while measuring two outcomes: prediction error, expressed as root mean squared error (RMSE), and single-sample prediction latency. Lower values are preferred for both, so improving accuracy can conflict with improving speed.

The study used a 140-run face-centered central composite design planned in advance and repeated across five data-split blocks. This structure separated large hyperparameter effects from smaller changes caused by random train/validation partitions. It detected strong curvature, identified interactions, and produced quadratic response surfaces for candidate selection. Those surfaces were informative but imperfect: validation-RMSE lack of fit was significant, and the most accuracy-focused boundary candidate was more optimistic in the model than in independent confirmation. A balanced depth-4 candidate confirmed within its prediction interval, while a depth-7 accuracy candidate missed its interval by 0.0088 RMSE.

In a later 20-replicate comparison, single-objective TPE achieved the lowest observed mean holdout RMSE (0.46691) but with higher latency (188.93 μs). Repeated DOE and multi-objective TPE had no statistically significant difference in holdout RMSE (Welch $p = 0.3826$) at similar latency; this non-significant result does not establish equivalence. The evidence supports a complementary interpretation: planned experiments are valuable for explanation, variance control, and stable structured selection, while adaptive search is effective at concentrating evaluations in promising regions. Results are limited to one dataset, four tuned factors, 100-tree XGBoost models, and a historically exposed holdout dataset.

Keywords: XGBoost, hyperparameter optimization, design of experiments, response surface methodology, prediction latency, multi-objective optimization.

Contents

1 Introduction: the problem we want to solve **3**

2 Our core idea: learning from experiments **4**

3 How we designed the experiment **6**

3.1 Data partitions and holdout history 6

3.2 Why blocking was necessary 6

3.3 What the 140-run design means 6

3.4 What we measured 7

4 How we learned from the experimental results **7**

4.1 Screening and curvature 7

4.2 How settings shaped accuracy 8

4.3 How model complexity affected speed 9

5 How we selected candidate models **9**

6 How we checked whether the findings held up **10**

7 Comparing DOE with alternative optimization methods **12**

8 What we learned **14**

9 Conclusion **15**

10 Code availability **15**

1 Introduction: the problem we want to solve

XGBoost builds a prediction model by adding decision trees one after another [2]. Several settings—called **hyperparameters**—control how quickly the model learns, how complex each tree may become, how much training data each tree sees, and how strongly the model is regularized. Choosing these settings is consequential: a configuration can be accurate but slow, fast but inaccurate, or sensitive to the particular random data split used during evaluation.

Our research question is:

> **Can a structured, pre-planned experimental design help us understand and optimize an XGBoost model, and how does that approach compare with adaptive optimization methods such as Optuna/TPE?**

We measured two practical objectives. **Root mean squared error (RMSE)** summarizes the typical size of prediction errors in the target's units, while placing extra weight on large errors; *lower is better*. **Inference latency** is the time required for one trained model to predict one observation, measured in microseconds; here too, *lower is better*. A deeper or more complex model may reduce RMSE while taking longer to evaluate, so there is rarely one configuration that is best on both axes.

Finding one strong configuration was only part of the question. We also wanted to know which settings mattered, whether settings interacted, how results changed across random data partitions, whether a simple mathematical approximation was adequate, and how the accuracy–speed trade-off changed the preferred model. A planned **design of experiments (DOE)** provides that explanatory structure [1]. Adaptive methods such as the **Tree-structured Parzen Estimator (TPE)** instead use earlier trials to decide where to search next [4, 5]. Neither strategy is assumed to be universally superior; they spend information differently.

Figure 1 shows the investigation from question to final comparison. The complete formulas, ANOVA tables, diagnostics, timing details, and provenance checks are retained in the companion technical supplement.

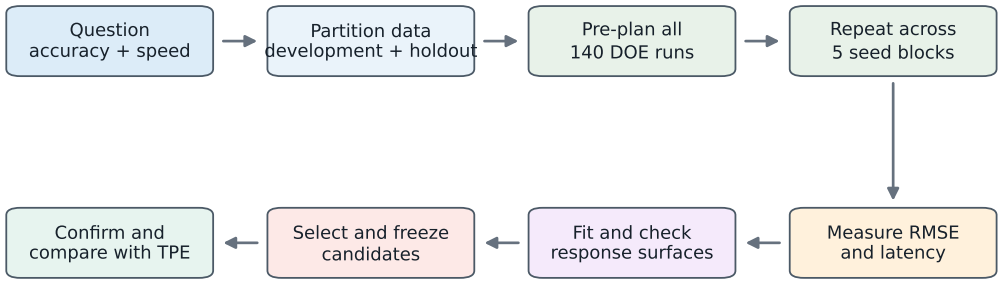


Figure 1: The research pipeline. All 140 historical FCCD runs were planned before execution and globally randomized. The labels "Phase 1" and "Phase 2" describe hierarchical analyses of that completed design; Phase 1 results did not decide whether later runs would be executed.

2 Our core idea: learning from experiments

An optimizer asks, "Where should I try next to find a good model?" A designed experiment also asks, "What can this collection of trials teach us about the system?" DOE chooses combinations so that changes in the response can be attributed to individual factors, interactions between factors, and curvature. **Response surface methodology (RSM)** then fits a low-order mathematical surface through those planned measurements [7, 8]. The surface is a local guide: it can summarize broad tendencies and suggest candidates, but it must be checked against observations before being trusted.

We varied four XGBoost hyperparameters while holding the ensemble size at 100 trees:

The face-centered central composite design (FCCD) combines factorial corners, axial points that move one factor at a time, and repeated center points. Figure 2 shows a two-factor projection. Corners reveal main effects and interactions, axial points help estimate curvature, and center replicates measure variation when the nominal configuration is unchanged.

Hyperparameter	Practical meaning	Studied range	Expected trade-off
Learning rate η	Size of each boosting update	0.01–0.30 (log scale)	Small values learn slowly; large values can converge faster but overshoot.
Maximum depth	Maximum tree complexity	3–9	Deeper trees can model complex patterns but cost more to train and predict.
Subsample	Fraction of training rows used per tree	0.50–1.00	Sampling can regularize the model but adds stochastic variation.
L_2 regularization λ	Penalty on leaf weights	0.10–10.00 (log scale)	Stronger shrinkage can reduce overfitting but may suppress useful structure.

Table 1: The four studied factors. Log scaling gives multiplicative changes in learning rate and regularization comparable spacing in the experimental design.

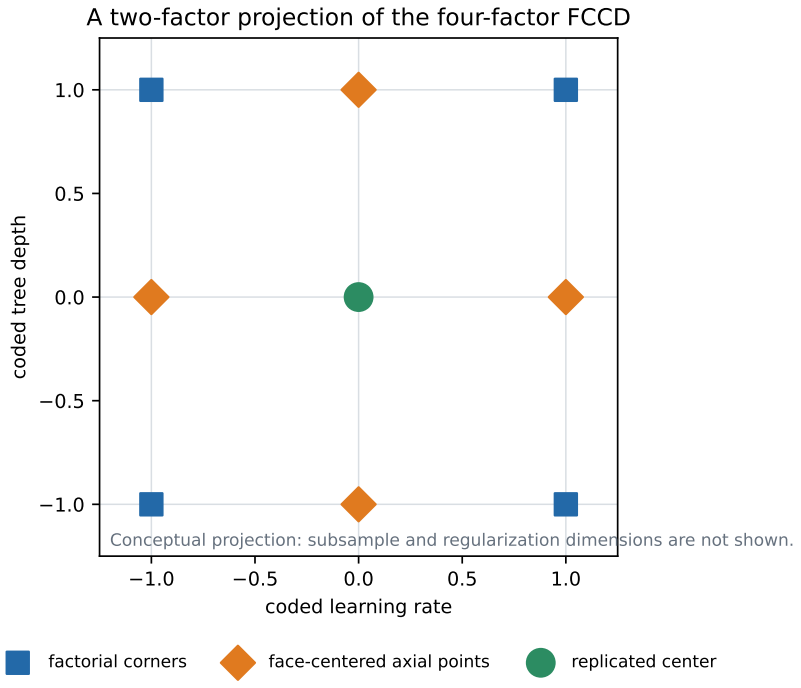


Figure 2: How the FCCD explores factor space. This simplified projection shows learning rate and depth only; the actual experiment used four dimensions. The same planned coordinate types were repeated across data-split blocks.

3 How we designed the experiment

3.1 Data partitions and holdout history

The California Housing dataset contains 20,640 observations. A fixed 20% external holdout contained 4,128 observations. The remaining 16,512 observations formed the development pool. Within each seed block, that pool was divided into 12,384 training and 4,128 validation observations. Search decisions used validation RMSE, while final generalization checks used the holdout test set.

During the repeated benchmark experiments, the selection procedure programmatically prevented access to holdout outcomes until the models were fixed for final evaluation. Earlier experiments had already reported results from the same holdout dataset, however. The holdout therefore cannot be described as untouched across the project's full history.

3.2 Why blocking was necessary

Changing a hyperparameter is not the only reason RMSE changes. A different random split can make a training problem slightly easier or harder. We therefore repeated planned configurations across five split seeds, called **blocks**. This allowed the statistical model to separate systematic hyperparameter effects from partition-related nuisance variation. Figure 3 shows center-point observations: the configuration is nominally the same, yet the block means move.

The estimated block standard deviation was approximately 0.0077 RMSE. Blocks accounted for an intraclass correlation of 0.4069 (40.7% of unexplained residual variance), matching the REML estimate 0.4069. This is not 40.7% of total response variation; the main factor effects dominate the wide design region. It is nevertheless large enough to matter when competitive configurations differ by only a few thousandths of RMSE.

3.3 What the 140-run design means

Each of five blocks contained 16 factorial corners, eight face-centered axial points, and four center runs, giving 140 model fits in total. All runs were generated in advance and globally randomized. In the shuffled execution order, 30 of the 40 axial runs happened before run 100. Thus the historical campaign was not an adaptive sequence in which an initial analysis triggered an augmentation.

The matched numerical budget needs qualification. DOE used 25 distinct coordinates across five blocks plus extra center replication; TPE and Random Search used 140 distinct configurations on one development split. DOE spent evaluations on variance estimation and structured effect identification. Adaptive search spent them on denser coordinate exploration. In addition, 45 of the DOE's 140 runs (32.14%) lay at $\eta = 0.01$, where 100 trees underfit: mean validation RMSE was 0.7545 there versus 0.5055 elsewhere.

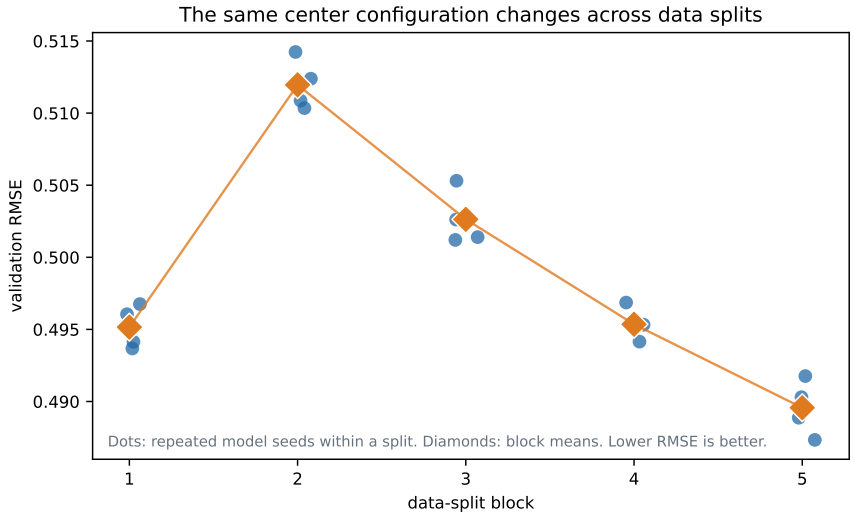


Figure 3: Why blocking matters. Canonical center-point measurements vary across data-split blocks even though the hyperparameters remain fixed. Repeated model seeds within a block show subsampling variation; movement between block means shows partition variation.

3.4 What we measured

For each trial we recorded development validation RMSE and single-row CPU prediction latency. The initial measurements used warmup and 1,000 timed calls per prediction interface. The repeated searches used a cheaper 30-call online measurement, while final verification used two warmed 500-call sessions. Standard `predict` also includes data-wrapper overhead: the interface comparison measured $32.19 \pm 1.46\,\mu$s more than `inplace_predict`. Search-time latency and benchmark verification latency are therefore distinct measurements and are never treated as interchangeable.

4 How we learned from the experimental results

4.1 Screening and curvature

The first analysis asked which effects were large enough to deserve attention. Learning rate dominated the first-order factorial model ($F = 357.40$), followed by depth ($F = 23.57$) and their interaction ($F = 26.32$). An interaction means the effect of one setting depends on another: increasing depth does not have one constant consequence at every learning rate.

Center runs then tested whether a flat plane was adequate. The factorial mean exceeded the center mean by 0.1343 RMSE (26.93%). Curvature was strong under every reported error denominator: $F = 105432.31$ against center pure error, $F = 20{,}235.41$ against pooled block-adjusted error, and $F = 7{,}342.69$ in the paired five-block analysis. The conclusion is non-planarity, not proof that a quadratic surface is globally correct. That distinction motivated the second-order fit and its adequacy tests.

4.2 How settings shaped accuracy

We wanted a mathematical picture of how learning rate and tree depth changed accuracy, rather than only a ranked list of configurations. Figure 4 shows a two-factor slice of the fitted quadratic surface. The horizontal axis is coded log learning rate, the vertical axis is coded maximum depth, and the contours and height show predicted validation RMSE; darker, lower regions are better. The surface falls sharply as learning rate moves away from the underfitting boundary, then forms a broad ridge. The cyan depth-7 candidate sits near the best in-domain region, while the stationary point indicated by the arrow lies outside the experimental domain. This geometry is why we optimized within the tested bounds and required confirmation.

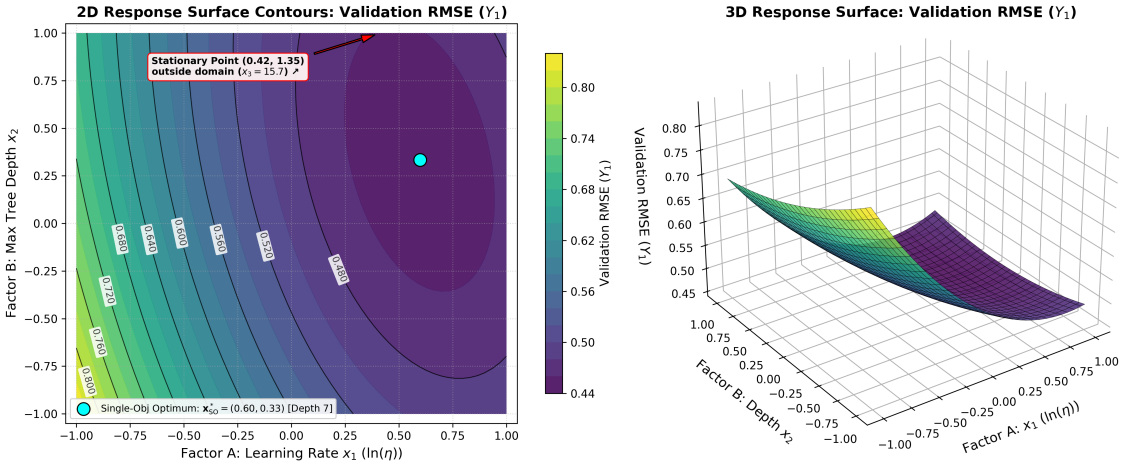


Figure 4: Empirical validation-RMSE response surface. Left: contours across coded log learning rate and depth at the single-objective slice. Right: the same fitted surface in three dimensions. The broad valley guides candidate selection, but the extrapolated stationary point lies outside the studied region.

The quadratic validation-RMSE model had $R^2 = 0.9951$ and adjusted $R^2 = 0.9944$. These high values mainly reflect large changes across the full learning-rate range. Formal lack-of-fit remained significant: $F = 62.21$ with $p = 7.78e - 41$, and the lack-of-fit RMS scale was 0.0297. When the severe $\eta = 0.01$ boundary was removed, lack-of-fit sum of squares fell by 96.3%, although the restricted model still did not justify treating the surface as exact.

The residual diagnostics, reproduced in the technical supplement, showed approximately normal residuals but also heteroscedasticity and localized influence. HC3 heteroscedasticity-consistent standard errors were used separately for coefficient inference; they do not remove model misspecification. Breusch–Pagan provided evidence of heteroscedasticity, while the serial diagnostics did not detect significant dependence in the tested residual order. Those diagnostics do not prove the absence of every temporal pattern or independently establish the absence of thermal drift.

4.3 How model complexity affected speed

The next question was whether deeper trees imposed a predictable latency cost. In Figure 5, the horizontal axis is maximum tree depth and the vertical axis is measured single-sample prediction time. The left panel compares empirical means and 95% intervals with linear and quadratic fits; the right panel shows the full distribution at each tested depth. Latency rises modestly from depth 3 to 6 and much more sharply by depth 9, so a straight line understates the acceleration.

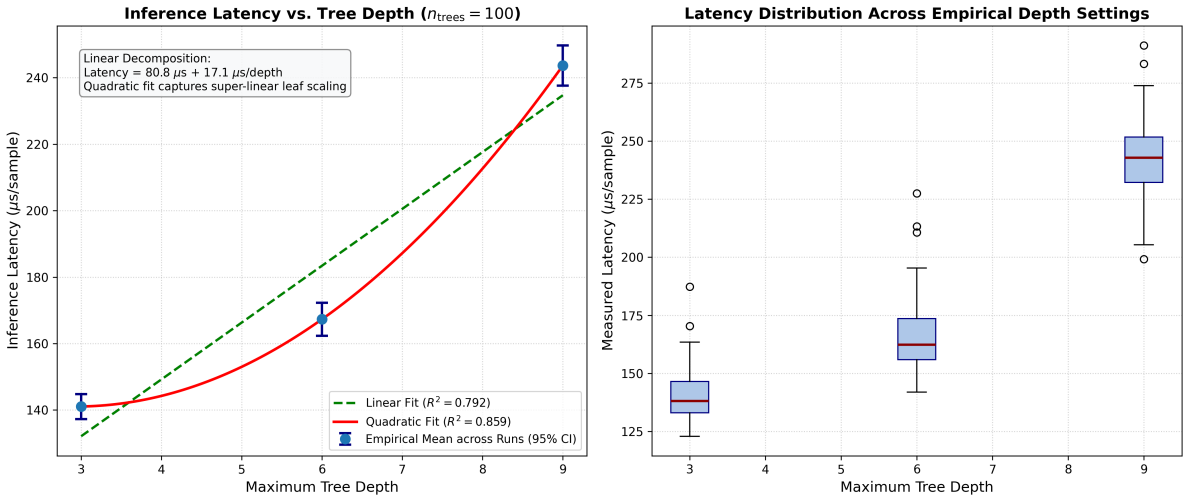


Figure 5: Measured prediction latency by maximum tree depth. The quadratic curve follows the accelerating cost better than the linear fit, while the boxplots show run-to-run variation around each tested depth.

A quadratic depth profile achieved $R^2 = 0.859$ compared with $R^2 = 0.792$ for a linear profile. Latency lack of fit was not significant ($F = 1.15$, $p = 0.3305$). This result applies to the tested hardware and single-sample protocol, not to batch throughput or all deployment systems. The observed acceleration made depth central to the later accuracy–speed compromise.

5 How we selected candidate models

The fitted surface had a nearly flat direction: its smallest canonical eigenvalue was 0.000057, with bootstrap interval $[-0.0039, 0.0022]$, and 68.2% of bootstrap replicates placed it at or below zero. The unconstrained stationary point lay far outside the studied region. We therefore interpret the geometry as a stationary or rising ridge rather than a well-defined bowl with one stable interior minimum.

We used two selection goals. The **single-objective** candidate minimized predicted validation RMSE and moved toward a depth-7 boundary solution. The **multi-objective** candidate combined RMSE and latency through a Derringer–Suich desirability score [6]: each outcome was converted to a preference between zero and one, then combined. This

selected an interior depth-4 compromise with learning rate 0.2324, subsample 1.0000, regularization 0.8295, and composite desirability 0.6782.

Figure 6 shows how that decision emerged from the planned experiment. The horizontal axis is five-block mean validation RMSE and the vertical axis is five-block mean latency; moving left or down is better. Pale points are individual block observations, colored circles are configuration means, and the dashed staircase is the observed Pareto front. A point is **Pareto efficient** when no observed alternative is better on both outcomes. The depth-7 diamond pursues accuracy, whereas the depth-4 star gives up a small amount of predicted accuracy for much lower latency. That trade-off motivated carrying both candidates into confirmation.

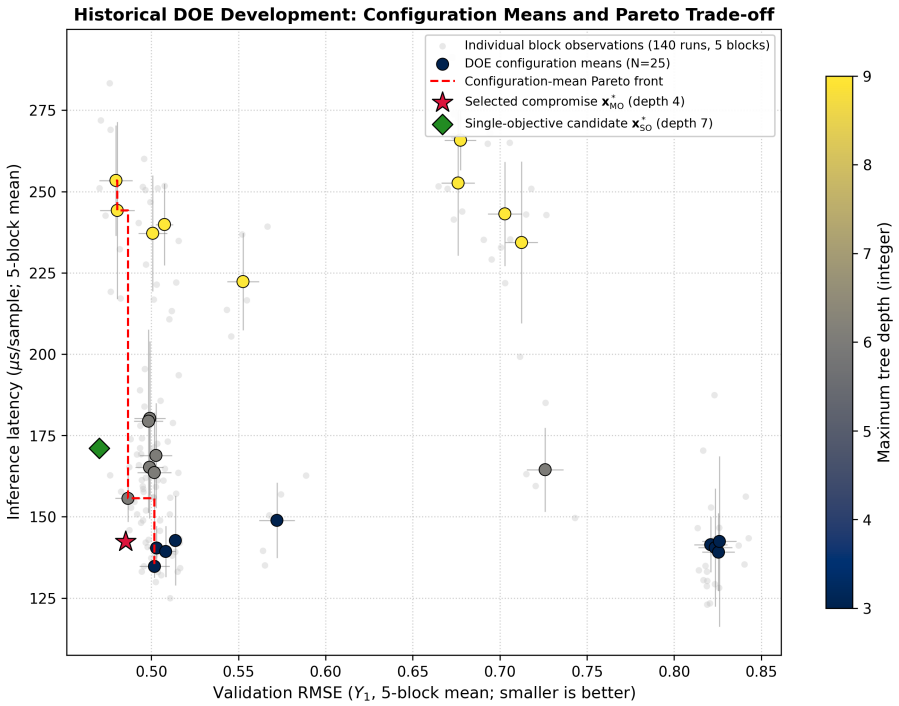


Figure 6: Accuracy–speed evidence from the initial blocked experiment. Configuration means and their between-block uncertainty reveal the Pareto trade-off used to select the depth-4 compromise and depth-7 accuracy candidate.

Sensitivity analysis changed desirability bounds and weights. The compromise moved as intended rather than producing one invariant optimum, which is why the desirability choice is treated as a declared decision rule rather than a discovered physical constant. Full sensitivity results remain in the technical supplement.

6 How we checked whether the findings held up

First, we evaluated the historical candidates across ten confirmation seeds:

$$\mathcal{S}_{\text{conf}} = \{505, 606, 707, 808, 909, 1010, 1111, 1212, 1313, 1414\}.$$

Configuration / Response Metric	Surrogate Pred ($\hat{y}$)	95% Pred Interval (PI)	Empirical Mean ± SD	Confirmation Status
DOE Multi-Objective Optimum $\mathbf{x}^*_{\mathbf{MO}}$ (Depth 4)				
Validation RMSE (Y_1)	0.4804	$[0.4677, 0.4932]$	0.4853 ± 0.0115	**Pass** (Inside 95% PI)
Holdout Test RMSE	—	—	0.4888 ± 0.0032	Holdout Test Set
Inference Latency (μs)	138.1	$[122.8, 153.4]$	142.3 ± 7.8	**Pass** (Inside 95% PI)
DOE Single-Objective Candidate $\mathbf{x}^*_{\mathbf{SO}}$ (Depth 7)				
Validation RMSE (Y_1)	0.4483	$[0.4354, 0.4612]$	0.4700 ± 0.0093	**Not Confirmed** (Optimism: +0.0217)
Holdout Test RMSE	—	—	0.4691 ± 0.0034	Holdout Test Set
Inference Latency (μs)	187.0	$[171.1, 202.9]$	171.1 ± 10.9	Borderline (At Lower PI)

Table 2: Independent confirmation of the historical multi-objective compromise and accuracy-focused boundary candidate. The compromise confirmed within its interval; the boundary candidate exposed polynomial optimism.

The intervals combined regression uncertainty with estimated block variance through a Satterthwaite approximation. This reflects uncertainty from a new partition, but it cannot guarantee protection against localized polynomial bias.

The depth-4 compromise produced validation RMSE 0.4853 ± 0.0115, inside its 95% interval $[0.4677, 0.4932]$. The depth-7 candidate produced 0.4700 ± 0.0093, above its upper interval bound by 0.0088 and showing optimism bias +0.0217. Confirmation therefore supported the compromise pointwise while warning against global trust in the surface.

For the repeated benchmark experiments, every selected configuration was fixed before final evaluation. Each selection was then retrained across 20 seeds from 2001 through 2020 and assessed on the external holdout. Repeated DOE used 20 fresh search replicates, each containing five development blocks. Its single-objective rule selected the same predeclared grid point in every replicate, so between-search SD was 0.00000; conditional retraining SD at that configuration was still 0.00304. These quantities describe different sources of variation and must not be conflated.

7 Comparing DOE with alternative optimization methods

We compared four adaptive or stochastic alternatives:

- **Random Search** samples configurations without learning from earlier results.
- **Single-Objective TPE** adapts its sampling to reduce validation RMSE.
- **Multi-Objective TPE** searches for configurations spanning the RMSE–latency trade-off.
- **Constrained TPE** minimizes RMSE while treating 145 μs as a latency constraint.

Table 3 explains why equal counts of model fits do not make the information identical. The complete campaign used 17,077 model fits. Each search replicate used 140 fits, but DOE replicated planned coordinates across splits while black-box methods explored distinct coordinates on one split. The technical supplement provides the detailed execution budget and timing ledger.

Method	**Use of the 140-fit budget**	**Noise handling during search**	**Primary information gained**
Blocked DOE	25 planned coordinates	Five data-split blocks	Effects, interactions, curvature, and split-to-split variation
Random Search	140 sampled coordinates	One development split	Broad non-adaptive exploration
TPE variants	140 adaptive coordinates	One development split	Concentrated optimization in promising regions

Table 3: What the compared methods did with the same 140-fit search budget. DOE emphasized estimable effects and split-to-split variation; Random Search emphasized broad coverage; TPE emphasized adaptive concentration. These are different experimental estimands even when fit counts match. Each search was repeated 20 times, and fixed selections were retrained across 20 evaluation seeds.

Before comparing final models, we asked how quickly each search found lower validation error. Figure 7 plots cumulative model fits on the horizontal axis and the best validation RMSE found so far on the vertical axis. Falling curves indicate improvement; shaded bands show the middle half of repeated runs. TPE improves rapidly because it adapts toward promising regions. The planned DOE trace improves in scheduled jumps because its run order was randomized for unbiased analysis rather than arranged for early optimization. By the end of the budget, several curves are close, but they reached that region in different ways and produced different kinds of information.

Optimization Method	**Search Basis** (N Reps)	**Val RMSE** Mean ± SD	**Holdout Test RMSE** Mean ± SD	**Retrain** σ_{eval}	**Latency** (μs ± SD)	**Feasible** ($\leq 145\,\mu$s)
Panel A: Repeated Benchmark Experiments ($N = 20$ Search Replicates)						
Repeated DOE MO ($\mathbf{x}^*_{\text{MO}}$)	RSM (N=20)	0.4853 ± 0.0055	0.4900 ± 0.0073	0.0033	122.3 ± 2.7	20/20
Multi-Objective TPE	Parzen (N=20)	0.4881 ± 0.0093	0.4926 ± 0.0108	0.0036	121.5 ± 3.8	20/20
Constrained TPE ($\leq 145\,\mu$s)	Parzen (N=20)	0.4700 ± 0.0022	0.4704 ± 0.0032	0.0034	144.3 ± 7.1	9/20
Repeated DOE SO (d=7)	RSM (N=20)	0.4687 ± 0.0000	0.4686 ± 0.0000	0.0030	150.7 ± 0.7	0/20
Single-Objective TPE	Parzen (N=20)	0.4675 ± 0.0008	0.4669 ± 0.0012	0.0031	188.9 ± 12.1	0/20
Unguided Random Search	Uniform (N=20)	0.4687 ± 0.0014	0.4695 ± 0.0021	0.0030	181.1 ± 19.0	0/20
Panel B: Initial Experiment Snapshot (Single Search Replicate)						
Historical DOE MO ($\mathbf{x}^*_{\text{MO}}$)	RSM (Single)	0.4821	0.4884	—	119.8	Yes
Historical Multi-Obj TPE	Parzen (Single)	0.4812	0.4870	—	118.8	Yes
Historical Constrained TPE	Parzen (Single)	0.4783	0.4823	—	125.4	Yes
Historical DOE SO (d=7)	RSM (Single)	0.4687	0.4690	—	147.8	No
Historical Bayesian TPE	Parzen (Single)	0.4735	0.4726	—	193.2	No
Historical Random Search	Uniform (Single)	0.4696	0.4707	—	198.1	No

Table 4: Final benchmark results across 20 repeated searches and 20 holdout retraining seeds. Initial single-campaign rows are retained for context and are not interchangeable with the repeated benchmark experiments.

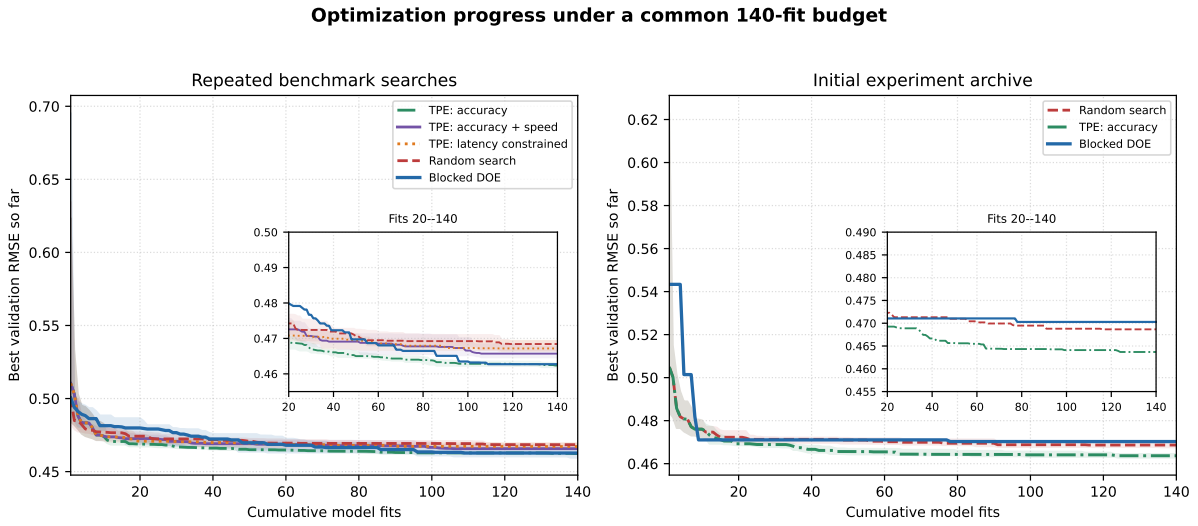


Figure 7: How optimization progressed over the common 140-fit budget. Left: medians and interquartile bands from 20 repeated searches per method. Right: the initial single planned experiment compared with archived random-search and TPE trajectories. Lower curves are better; the insets magnify later differences.

Single-objective TPE achieved the lowest observed mean holdout RMSE, 0.46691 ± 0.00116, at $188.93 \pm 12.07\,\mu$s. Repeated DOE single-objective achieved 0.46859 at $150.74\,\mu$s, which was 20.2% lower latency. Neither accuracy-only method satisfied the latency constraint.

For multi-objective selection, Repeated DOE achieved 0.48997 ± 0.00732 RMSE at

122.29 μs; multi-objective TPE achieved 0.49255 ± 0.01080 at 121.46 μs. The primary unpaired Welch comparison gave $\Delta = -0.00258$, $t = -0.8849$, and $p = 0.3826$. Replicate indices did not share a random block, so an index-paired result is only a secondary sensitivity check. No equivalence margin was pre-registered: failure to detect a difference is not evidence of equivalence.

Constrained TPE appeared feasible in 20/20 online searches but only 9/20 (45%) remained feasible under warmed 1,000-call verification. Depth-6 selections were feasible at 136.70 μs; depth-7 selections were not, averaging 150.45 μs. Short noisy timing, protocol differences, and selection near the threshold are plausible explanations.

The final question was what the competing selected models achieved after retraining. In Figure 8, the left panel plots search-time validation RMSE against online latency for development candidates; the right plots holdout RMSE against verified latency for the fixed selections. Horizontal and vertical uncertainty bars on the right show retraining and timing variation. The red dashed frontier identifies choices for which no tested alternative was both more accurate and faster.

On development split 42, multi-objective TPE candidate fronts had mean hypervolume 18.3834 ± 0.3301 at reference $[0.60, 250]$, compared with 15.6128 ± 0.7268 for five-block repeated DOE means and 16.8585 for the fixed 27-point DOE front. These rows use different candidate pools and timing protocols. On the holdout, 12 distinct configurations formed the non-dominated set with frozen hypervolume 17.6714; seed-bootstrap mean was 17.6923 with 95% interval $[17.5848, 17.8061]$. The secondary reference point was $[0.65, 275]$.

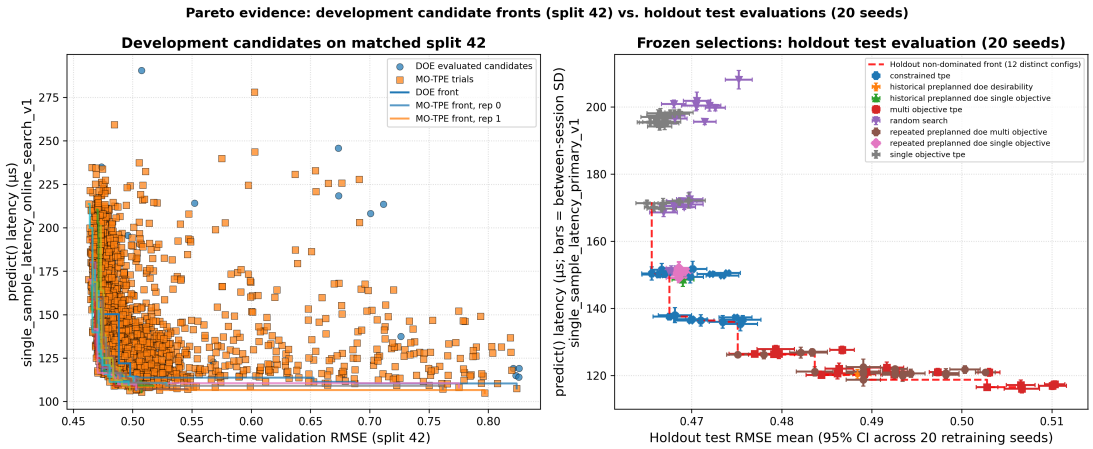


Figure 8: Canonical Pareto evidence. Left: development candidate fronts under their stated search protocols. Right: frozen selections after independent holdout evaluation, including uncertainty. The figure separates development search measurements from final benchmark measurements.

8 What we learned

1. **Structured DOE explains variation well.** It identified dominant factors and interactions, quantified curvature, and separated an estimated split-related nuisance

component from treatment effects.

2. **A response surface is a guide, not ground truth.** Strong global R^2 coexisted with significant local lack of fit and a failed confirmation interval at the boundary candidate.
3. **Adaptive search used the budget effectively for point optimization.** Single-objective TPE found the lowest observed RMSE, and multi-objective TPE produced strong development fronts by concentrating trials in promising regions.
4. **The deployment choice depends on the accuracy–speed objective.** The fastest acceptable and most accurate configurations were different; the holdout frontier contained selections from DOE and several TPE variants.
5. **Latency constraints require careful verification.** Constrained TPE's feasibility dropped from search to warmed benchmarking, showing that a noisy online timing estimate is insufficient near a hard threshold.
6. **The comparison remains limited.** DOE and TPE used equal fit counts but different coordinate replication, split structures, and timing protocols. The fixed holdout does not estimate dataset-sampling uncertainty, and prior historical exposure prevents describing it as untouched. Results come from one dataset, one hardware environment, four factors, and 100 trees. Stronger claims require fresh outer test splits or additional datasets, matched timing protocols, more model families, and designs that include the number of trees.

9 Conclusion

A pre-planned blocked experiment can do more than produce a candidate configuration: it can show which hyperparameters matter, expose interactions and curvature, quantify random split effects, and reveal when its own approximation is inadequate. In this study, that explanatory value coexisted with clear limitations. The quadratic surface was not globally faithful, the accuracy-focused DOE candidate was optimistic in confirmation, and adaptive TPE found the lowest observed single-objective RMSE.

The multi-objective evidence does not identify one universal winner. Repeated DOE and multi-objective TPE produced similar observed holdout performance at comparable latency, with no statistically significant detected difference and no basis for an equivalence claim. The practical lesson is to combine the strengths of both approaches: use structured experiments when explanation, variance decomposition, and stable comparison matter; use adaptive search when dense exploitation of promising regions is the main goal; and independently confirm every selected model under the deployment measurement protocol.

10 Code availability

The code and research materials are publicly available at: https://github.com/ChristosHussein/xgboost-doe-rsm

Companion technical supplement

The complete derivations, generated ANOVA and adequacy tables, residual diagnostics, timing forensics, hypervolume details, computational budget, and claim-level provenance are provided in `technical_supplement.pdf`. The comprehensive numerical record remains in `REPORT.md`.

References

[1] D. C. Montgomery, *Design and Analysis of Experiments*, 9th ed., Wiley, 2017.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *KDD*, 2016.

[3] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *JMLR*, 2012.

[4] J. Bergstra et al., "Algorithms for hyper-parameter optimization," *NeurIPS*, 2011.

[5] T. Akiba et al., "Optuna: A next-generation hyperparameter optimization framework," *KDD*, 2019.

[6] G. Derringer and R. Suich, "Simultaneous optimization of several response variables," *Journal of Quality Technology*, 1980.

[7] G. E. P. Box and K. B. Wilson, "On the experimental attainment of optimum conditions," *JRSS B*, 1951.

[8] R. H. Myers, D. C. Montgomery, and C. M. Anderson-Cook, *Response Surface Methodology*, 4th ed., Wiley, 2016.